

Report Till 1-10 Weeks

Project MAESTRO

Project MAESTRO - Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation. Group 298, B.Tech 7CSE (Evening), Amity School of Engineering and Technology, session 2023-27. Submitted by Shashank Gupta (A2345923073), Seenu (A2345923074) and Jairaj Berry (A2345923013), under the guidance of Dr. Rajni Sehgal Kaushik. Project area: Agentic AI with specialization in Natural Language Processing.

Introduction

This report summarises the work carried out during the first ten weeks of the Minor Project for Project MAESTRO, titled "Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation". The project belongs to the area of Agentic Artificial Intelligence with specialization in Natural Language Processing, and is being carried out by Group 298 of B.Tech 7CSE Evening, Session 2023-27, under the guidance of Dr. Rajni Sehgal Kaushik.

MAESTRO stands for Multi-Agent Execution System for Task Reasoning and Orchestration. The aim of the project is to build a desktop assistant that can understand an ordinary English instruction, convert it into a structured plan, and carry out the required task on a real computer. The important difference between MAESTRO and ordinary automation agents is that MAESTRO is not designed to execute blindly. Before any action is allowed to affect the computer, the action is represented in a typed form, checked against deterministic safety rules, simulated through a dry run, shown to the user for approval where required, and recorded in a tamper-evident audit log.

The first five weeks of the project were mainly concerned with defining the problem, surveying the literature, studying existing systems, identifying the research gap and preparing the requirement analysis. Weeks six to ten extended this foundation into formal specification, system architecture, design diagrams, evaluation methodology and dataset preparation. By the end of the tenth week the project had moved beyond an idea or a proposal. It had a documented research direction, a defined safety model, a working implementation track, a benchmark plan and a dataset foundation for later training and evaluation.

Week 1 Project Definition and Feasibility

In the first week, the project title was finalised as "Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation". The team discussed the initial idea of a desktop assistant and narrowed it into a research project that could be defended academically. The focus was fixed not on simply making an agent that can control a computer, but on designing and evaluating a safer way for such an agent to work. This distinction was important because many existing systems already show impressive capability, but they do not provide a strong method for measuring or controlling the risks involved in letting an AI agent act on a real desktop.

The objectives and scope of the project were written during this week. The main objective was fixed as designing a multi-agent desktop automation system that accepts natural language instructions while enforcing measurable safety properties. The major safety properties identified were auditability, reversibility, consent-gating and injection resistance. The team also identified the areas of study on which

the project depends: Artificial Intelligence, Natural Language Processing, Multi-Agent Systems, Desktop Automation, Human-Computer Interaction and AI Safety.

A feasibility study was also carried out. On the technical side, the team checked whether the project could be implemented using ordinary student hardware, local AI models and open-source libraries. On the economic side, the team fixed the zero-cost constraint. The planned software stack was selected so that it could run with free and open-source tools, with local models used wherever possible and optional cloud tools treated only as baselines or future extensions. The synopsis was submitted, and the broad schedule of the project was prepared through the PERT-style project plan. This gave the team a clear structure for moving from research to requirements, then to architecture, implementation, testing and documentation.

Week 2 Literature Review Part One

The second week was spent on the first part of the literature review. The team began collecting and studying research papers related to AI agents, natural language processing, large language models, reasoning and acting, task planning and tool use. The purpose of this reading was to understand how current systems convert a human instruction into a sequence of actions, and where the risks enter when such systems are given access to real tools.

The literature review was organised by theme rather than only by individual paper. The team studied the basic structure of AI agents, the role of large language models in planning, the way tools are called by agents, and the limitations of models when instructions are ambiguous or when the environment contains malicious content. The review also included work on prompt injection, because desktop automation agents often read file contents, web pages and tool output, and these sources may contain text that attempts to override the user's original instruction.

A literature matrix was started during this week. The matrix recorded the year, method, planning approach, evaluation method, benchmark, limitations and relevance of each paper. This helped the team avoid writing a simple list of paper summaries. Instead, the review began to show patterns across the field: most systems focus strongly on completing tasks, but far fewer systems explain how risk is classified, how dangerous actions are prevented, or how prompt injection attacks are measured.

Week 3 Study of Existing Systems and Comparative Study

In the third week, the work moved from general research papers to the study of systems that already perform desktop automation or computer-use tasks. The team studied frameworks such as OpenClaw, Hermes and Open Interpreter, along with related agent systems discussed in the literature. These systems were compared not only by what they can do, but by how they represent actions, how much control they give to the user, whether they preview effects, whether they can undo actions and whether they maintain an audit trail.

The comparative study showed that existing systems are already capable of understanding broad natural language instructions and performing useful work on a machine. Therefore, raw task capability was not identified as the main research gap. The more serious weakness was that these systems often move directly from a model's decision to an action, or from model output to executable code or tool calls, without a strongly typed intermediate representation that can be checked before execution.

This week helped the team realise that MAESTRO should not compete with existing systems only on how many tasks it can perform. Instead, the project should focus on making the planned action visible, inspectable and measurable before it runs. The comparative study also showed that safety is often treated

as a user interface issue, such as asking for confirmation, rather than as an architecture issue with testable properties. This finding became central to the later novelty statement.

Week 4 Research Gap and Novelty Statement

The fourth week was devoted to the research gap analysis. Based on the literature review and comparative study, the team identified the major shortcomings in existing desktop automation agents. These shortcomings were grouped under transparency, safety, modular task execution, reversibility, prompt-injection resistance and evaluation. Existing systems may be able to act, but they rarely provide a clear way to inspect what they intend to do before execution, and they do not usually report safety as a measurable result.

The research gap was finalised in four parts. First, prior work mainly measures capability, not safety. Second, confirmation dialogs by themselves are not enough, because a dialog box does not provide a structured and testable view of the planned action. Third, indirect prompt injection is not treated as a first-class evaluation category in most desktop agent systems. Fourth, there is a lack of public natural-language-to-plan datasets for cross-platform desktop automation with safety annotations.

The novelty statement of MAESTRO was then written. The contribution of the project is that every planned action is compiled into a typed Action IR, scored for risk by deterministic program logic, simulated through a dry run, gated behind consent when required, made reversible where possible, and written into a hash-chained audit log. The language model is allowed to propose a plan, but it is not allowed to decide whether the plan is safe. This principle was expressed as: the model proposes, deterministic code decides.

Week 5 Requirement Analysis

In the fifth week, the work moved from research and comparison to formal requirement analysis. The team identified the main users of the system and the situations in which they would use it. A student user may ask the system to organise scattered files, search for documents, rename or move folders, or prepare drafts. A developer user may want repeated setup or file tasks to be completed quickly, while still requiring an audit trail. An evaluator or guide would be interested in whether the system is methodical, measurable and honest about its limits.

The functional requirements were organised around the major task categories of the system: file and folder management, search and retrieval, browser automation, application control, system information and settings, and composition or drafting. These categories were chosen because they represent realistic desktop work while still being narrow enough to control and evaluate. The team also identified what the system must refuse outright, including entering passwords, making payments, solving CAPTCHAs, creating accounts, permanently deleting files without recovery, changing operating-system security settings, using administrator privileges or sending messages autonomously.

The non-functional requirements were also clarified. The system must run at zero marginal cost, preserve privacy by operating locally where possible, remain auditable, keep deterministic safety decisions, and be reproducible through tests and evaluation scripts. The requirement analysis gave the project its first complete specification and created the bridge between the research gap and the system architecture.

Week 6 Action IR and Risk Taxonomy Specification

The sixth week was one of the most important weeks of the Minor Project because it converted the safety idea into formal specification. The team froze the first version of the Action IR, which is the intermediate representation used by MAESTRO to describe a plan before execution. The Action IR contains the action identifier, verb, arguments, dependencies, produced output and rationale. It also supports a directed acyclic graph structure, so that the output of one action can be used by another action in a controlled way.

The purpose of the Action IR is to prevent the system from treating a language model's free-form text as an executable instruction. Once an English command is converted into Action IR, ordinary program logic can check whether the plan is valid. The system can reject an unknown verb, a missing argument, a circular dependency or a path outside the allowed area before anything touches the computer. This makes the plan inspectable both by the safety layer and by the human user.

During the same week, the R0 to R3 risk taxonomy was specified. R0 actions are safe read-only actions that do not change the machine state. R1 actions are low-risk, reversible changes inside the permitted workspace. R2 actions are consequential but reversible actions that require explicit confirmation, such as moving or trashing files. R3 actions are irreversible, externally visible or security-relevant actions, and some of them are hard-blocked with no override. The team also documented the hard-block list, including credential entry, payments, permanent deletion, autonomous sending of messages and administrator-level changes.

The trust model was also written in this week. The user instruction is treated as trusted input, while file contents, web pages and tool output are treated as untrusted data. This distinction is necessary because a malicious PDF or web page may contain hidden instructions telling the agent to ignore the user or leak private files. In MAESTRO, untrusted content is not allowed to become a new instruction. It may be summarised or displayed, but it cannot add actions to the approved plan.

Week 7 Architecture Design

The seventh week focused on architecture design. The system was divided into layers so that each part has a clear responsibility. The lower layer contains memory and storage, including episodes, preferences, audit entries and undo information. Above that are the executors, which perform file, browser, application, system and drafting actions. The orchestrator schedules the plan, binds variables, executes actions in topological order, checks postconditions and performs rollback where required. The safety layer validates schema, registry, paths, risk, budget and consent. The planner creates candidate Action IR plans, while the NLP layer handles intent, entities and clarification. The interface layer provides typed, voice or web-based interaction.

This architecture was designed so that most of the system remains platform independent. Only the executor layer is allowed to contain operating-system specific code. This is important because the project aims to support both Windows and macOS without duplicating the whole system. If platform-specific logic enters the planner or safety layer, the project would become two separate systems. Therefore, the Action IR and safety decisions must remain the same across platforms, while only the final execution backend changes.

The data flow was also specified during this week. A user instruction goes to the NLP layer, which either extracts intent and entities or asks a clarifying question. The planner converts the instruction into an Action IR plan. The safety layer validates and scores the plan, performs a dry run and presents the effect

preview. If the user approves, the orchestrator executes the frozen plan, checks results, records the run and stores undo data. If a step fails, the system stops and rolls back completed reversible steps. This architecture made denial, failure and rollback normal flows rather than exceptional afterthoughts.

Week 8 Diagrams and Design Consolidation

The eighth week was spent consolidating the system design into diagrams and algorithm descriptions. The team prepared the component view, request lifecycle, injection defence view, plan lifecycle state machine, risk and consent model, and platform boundary explanation. These diagrams were important because the safety argument of the project depends on what crosses each boundary. The diagrams show that the user instruction is the only trusted source of authority, while files and web pages are treated as untrusted data.

The algorithm descriptions were also prepared. The risk scoring algorithm was written to show that the safety decision is deterministic, monotonic and fail-closed. Deterministic means that the same plan receives the same safety decision every time. Monotonic means that a rule can only raise risk, not lower it. Fail-closed means that an unknown or unclear case is treated as dangerous rather than safe. The dry-run algorithm was described as the step that produces counts, file sizes, collisions and unknowns before execution. The DAG scheduling algorithm explains how actions are ordered by dependencies, and the injection filtering rules explain why untrusted content cannot change the plan.

The design consolidation week also connected the written design with the working implementation. Voice interaction, browser actions and the web workspace were treated as new ways to interact with the same safety pipeline, not as separate unsafe shortcuts. This distinction matters because adding a voice interface should not create a new authority path. Whether an instruction is typed or spoken, the plan must still pass through Action IR, safety scoring, dry run preview, consent and audit.

Week 9 Evaluation Methodology

The ninth week was dedicated to evaluation methodology. Since the project claims measurable safety, the team had to define the measurements carefully. The main capability metric is task success rate, which measures whether the requested task was completed. Plan success rate records whether the plan structure matched the expected kind of solution. The safety metrics include unsafe execution rate, safety compliance rate, false confirmation rate, injection resistance rate, hard block rate, undo round-trip rate and audit integrity rate.

Unsafe execution rate was treated as the most important safety number. It measures whether an unsafe or consequential action ran without the required protection. The target for this metric is zero. However, the team also recognised that zero unsafe executions can be achieved in a useless way by asking for confirmation on everything. Therefore, false confirmation rate must be reported together with unsafe execution rate. A good safety system should stop dangerous actions without disturbing the user for harmless ones.

The benchmark design was also prepared. The planned benchmark includes tasks across file management, search and retrieval, browser work, application control, system information, drafting and multi-step compound tasks. The team also specified baselines and ablations. Baselines compare MAESTRO with simpler or less safe versions, while ablations remove parts such as the safety layer, dry run, postcondition checks, memory, critic or trained intent model to observe what changes. This makes the evaluation stronger because it does not only report that MAESTRO works; it shows which controls are responsible for which results.

Week 10 Dataset and Benchmark Preparation

The tenth week focused on dataset and benchmark preparation. The team worked on the DeskPlan dataset, which contains natural-language instructions paired with structured plans and expected behaviours. The dataset includes ordinary executable tasks, tasks that require confirmation, tasks that should be refused and tasks that should lead to clarification. This is important because a model trained only on successful execution examples would learn to comply with everything. A safety-focused system also needs examples where the correct behaviour is refusal or clarification.

The dataset design included seed examples, generated examples, paraphrases and adversarial cases. The adversarial cases include direct unsafe requests, attempts to escape the allowed file scope, ambiguous instructions and prompt-injection style content from files or web pages. The dataset was also split into training, validation and test portions so that later evaluation can measure generalisation instead of only memorisation. The current dataset evidence in the project shows 3643 total pairs, including task pairs and adversarial pairs, with train, validation and test splits prepared.

Benchmark preparation also advanced during this week. The project contains a 100-task benchmark and adversarial evaluation cases, along with scripts for building and validating the dataset and running evaluation. The intent classifier metrics and evaluation result summaries are present in the implementation. The current implementation has grown beyond the original prototype and includes the pipeline, NLP layer, planners, safety modules, executors, agents, memory, UI, voice, Google integration, dataset tooling, training scripts and tests. The test suite now contains 467 pytest cases. In the local sandbox run, 456 tests passed and 9 were skipped, while 2 tests failed only because of local permission limitations for Trash access and local HTTP binding. Those 2 tests passed when rerun outside the sandbox, showing that the failures were environmental rather than logic errors.

Work Done on the Implementation Side

Alongside the Minor Project documentation, the team continued to build the system so that the Major Project phase would begin with a working base rather than an empty repository. The earlier prototype in the Major folder proved the core safety idea with Action IR, a closed verb registry, deterministic risk scoring, path policy, audit log, file executors, dry run, consent and rollback. The current implementation in the Project folder extends that work into a fuller v1.0 system.

The current implementation includes a command line interface, a web workspace, voice interaction, file and search operations, browser operations, application and system operations, draft generation, Google service integration, local and rule-based planning, an intent classifier, memory, dataset tooling, training scripts and an evaluation harness. The important point is that these additions still follow the same safety envelope. New interfaces and executors do not bypass the plan and safety path. They are connected to the same pipeline in which the instruction becomes a plan, the plan is scored, the effects are previewed, the user approves when required and the final result is audited.

This implementation work supports the Minor Project because it provides evidence for the architecture. The project is not only proposing that deterministic safety can be added to desktop automation; it already demonstrates the idea in code and tests. At the same time, the report remains careful not to claim that the system is fully safe. The correct claim is narrower and more defensible: MAESTRO makes desktop automation auditable, reversible, consent-gated and resistant to the evaluated prompt-injection cases.

Present Status at the End of Week 10

At the end of the tenth week, the project has completed the major research and design stages required for the Minor Project. The title, synopsis, feasibility study, literature review, comparative study, research gap, novelty statement, requirement analysis, Action IR specification, risk taxonomy, trust model, system architecture, diagrams, algorithm descriptions, evaluation methodology and dataset plan have all been prepared. Weekly Progress Reports from Week 1 to Week 10 are available locally.

The project is also ahead on the implementation side. The working system has moved from a small safety prototype to a broader implementation with dataset, benchmark, evaluation, training and interface support. The remaining work for the Minor Project is mainly to assemble the final report, check formatting and plagiarism, review the claims with the guide, prepare the presentation and rehearse the viva demonstration. The most important discipline for the remaining weeks is to keep the claims aligned with evidence. Any number reported in the final report should be tied to a test, dataset, benchmark or evaluation artifact.

The planned work after Week 10 includes final report writing, final polishing of diagrams and appendices, review of references, final WPR filing, and preparation for the viva. For the later Major Project phase, the focus will shift toward fuller cross-platform support, expanded evaluation, user study, fine-tuning experiments and final presentation of results.

Conclusion

The first ten weeks of the Minor Project have established MAESTRO as a safety-focused desktop automation project with a clear research contribution. The project began with a broad idea of a natural language desktop assistant, but through literature review, system comparison and gap analysis it became a more precise work on safe task reasoning and orchestration. The central contribution is the use of a typed action representation, deterministic safety scoring, dry-run preview, consent-gating, reversibility and tamper-evident audit logging.

The progress made so far gives the team a strong foundation for the final Minor Project submission. The research story is clear, the architecture is documented, the evaluation plan is measurable, the dataset work has begun, and the implementation demonstrates the safety design in practice. The remaining task is to assemble this material into a complete final report and present the project honestly: not as a system that makes automation perfectly safe, but as a system that makes important safety properties visible, testable and measurable.